

17 Day 9: CSS Grid 布局 + Git 远程仓库首次配置

🎯 最终交付成果

成果	验证方式
理解 Grid 容器、轨道（行/列）、间距的概念	能口头解释 Grid 和 Flexbox 的核心区别
用 CSS Grid 重构项目卡片区域	视觉效果与之前的 Flexbox 版本类似，但源码用 <code>display: grid</code> 实现
卡片布局能根据屏幕宽度自动调整列数	缩放浏览器窗口，卡片列数从 3 列自动变为 2 列、1 列
本地仓库关联到 GitHub 远程仓库，并完成首次推送	GitHub 上能看到完整的项目文件和提交历史

📁 最终文件结构

```
text

my-portfolio/
├── index.html
├── about.html
├── styles.css    ← 今天修改：项目卡片区域改用 Grid
└── script.js
```

新增： 本地仓库 `my-portfolio` 将关联到远程仓库 `https://github.com/<你的用户名>/my-portfolio`

📅 Step 架构总览

Step	内容	预计用时	目标
Step 1	晨会：回顾 + 目标 + 概念预热	10min	理解 Grid 的定位和意义
Step 2	Grid 核心概念（容器/轨道/间距）	15min	掌握 <code>display: grid</code> 、 <code>grid-template-columns</code>
Step 3	用 Grid 重构项目卡片区域	15min	替代原有 Flexbox 网格写法
Step 4	响应式列数（ <code>auto-fit</code> + <code>minmax</code> ）	15min	一行代码实现自适应列数，无需写多个 <code>@media</code>
Step 5	Flexbox vs Grid 对比总结	10min	明确一维布局 vs 二维布局的适用场景
Step 6	Git 远程仓库配置 + 首次推送	10min	掌握 <code>git remote add</code> 、 <code>git push -u</code>
Step 7	Code Review + 自评	10min	检验理解

🇨🇳 Step 1：晨会

昨日回顾（2分钟）

你昨天完成了：数组方法进阶（push/pop/map/filter/reduce）、用对象数组 + map 生成 HTML 卡片、`innerHTML` 插入页面、动态页脚年份（防御性编程）。

快速自测：

- 1. `map()` 和 `push()` 的核心区别是什么？
- 2. `.join("")` 在数据驱动渲染里起什么作用？

今日目标说明（3分钟）

目标	完成标志
理解 CSS Grid 的核心概念	能说出容器、轨道（行/列）分别是什么
用 Grid 重构项目卡片区域	卡片布局效果不变，代码换成 Grid 实现
掌握响应式列数写法	用 <code>auto-fit</code> + <code>minmax</code> 实现自适应，不用写多条 <code>@media</code>
完成 Git 远程仓库首次配置	代码成功推送到 GitHub，能在网页上看到

概念预热 (3分钟)

问题：你已经用 Flexbox 实现过项目卡片的排列。Flexbox 能控制"横向排列、自动换行"，但如果你想让卡片**严格对齐成整齐的行列**（比如每行 3 张，行与行之间的卡片高度、间距完全对齐），Flexbox 做起来会不会有点别扭？你觉得原因是什么？

PM 视角导入 (2分钟)

text

Flexbox：擅长"一维"布局—要么管一行，要么管一列，不能同时精确控制行和列

Grid：擅长"二维"布局—同时控制行和列，做整齐的网格、仪表盘、图片墙特别顺手

现实场景：

- 导航栏、按钮组 → 用 Flexbox（一维排列）
- 商品列表、图片画廊、后台仪表盘 → 用 Grid（二维网格）

另外，今天要第一次把代码推送到 GitHub 远程仓库：

- 本地仓库只在你自己电脑上，电脑丢了/硬盘坏了代码就没了
- 推送到远程仓库后，代码有了云端备份，也是以后部署上线、多人协作的基础

课前预习：

1. 想好两个自测题的答案
2. 想好"Flexbox 做整齐网格为什么别扭"的直觉回答
3. 确认你已有 GitHub 账号（如果没有，先去 github.com 注册一个，今天 Step 6 会用到）
4. 准备好了，回复"进入 Step 2"